A Beginner's Tutorial: the new HiLiftAeroML dataset with code to train a ML surrogate

New aerodynamics dataset released!



JUSTIN HODGES, PHD

MAY 21, 2026 PAID

HiLiftAeroML is a *brand new dataset* made public only yesterday. We're going to train some models to it and share the code below. When I gained access to it I knew I wanted to speedrun a tutorial and post about in <24 hours.

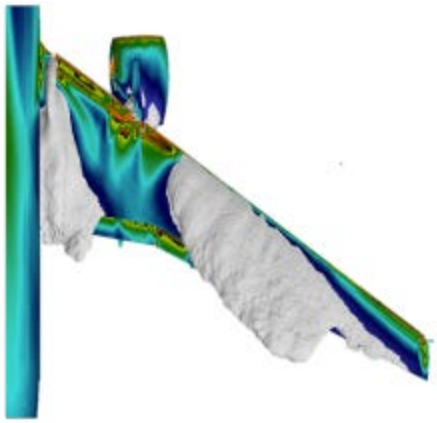


Image source here

This is an unusually important dataset for aerodynamic surrogate modeling because it brings high-fidelity, open CFD data to a problem class that has historically been studied through benchmark workshops, solver comparisons, turbulence-model studies, and validation campaigns.The underlying NASA High-Lift Common Research Model, or CRM-HL, has been central to the AIAA CFD High-Lift Prediction Workshop series, where standardized geometries, grids, and test cases

are used to assess CFD methods for deployed-flap/slat transport aircraft configurations. NASA describes these workshops as benchmarks for verifying and validating CFD methods on complex high-lift systems.

Before HiLiftAeroML, the CRM-HL ecosystem was mainly a CFD validation and methods-development platform. For example, NASA LAVA studies used CRM-HL to examine RANS sensitivity to mesh resolution, turbulence closure, tunnel effects, and near-CLmax behavior. HiLiftAeroML changes the situation by packaging 1,800 high-fidelity WMLES simulations across 180 geometry variants and 10 angles of attack, with forces, geometry definitions, surface fields, volume fields, and visual diagnostics available under an open license.

This blog walks through two deliberately small examples:

1. A **0D KPI surrogate** that predicts integrated aerodynamic coefficients: CL, CD, and CM.
2. A **3D point-cloud surrogate** that predicts a scalar field over sampled surface points.

The goal is not to build the most accurate possible model here. The goal is to make the dataset approachable, show how a Transolver++-style model can be connected to it, and give beginners a clean path from “I can run the notebook” to “I can turn this into a serious research or portfolio project.”

The Dataset

The HiLiftAeroML dataset here contains simulations of the NASA CRM-HL across geometry and angle-of-attack variation. The dataset includes lightweight CSV summaries as well as very large field data:

- force_mom_all.csv: integrated lift, drag, moment, pressure, and viscous coefficients

- geo_values_all.csv: design variables defining the geometry variants
- per-run boundary_*.vtu.tgz: time-averaged surface fields
- per-run volume_*.vtu.tgz: time-averaged volume fields
- STL/STP geometry files and rendered wall-shear visualizations

The important practical detail is file size. The coefficient CSVs are tiny and easy to use in Google Colab (free compute). The boundary and volume field archives are not: the dataset card lists boundary archives at roughly 13 GB compressed per case and volume archives at roughly 23 GB compressed per case. That strongly affects how we design beginner examples.

Model Choice: Why Transolver++

The notebooks use Transolver++, a neural PDE solver architecture designed for large, irregular geometries. Its appeal here is conceptual as much as practical: CFD data often lives on meshes or point clouds, not on neat image grids. Transolver++ uses a physics-attention mechanism that aggregates information from many mesh points into learned physical states, making it a natural family of models to try on surface and volume CFD fields. However, of course, you can use a few other top-tier models just fine in replacement to Transolver++.

In these notebooks, I keep the model small and Colab-friendly. That means the examples borrow the model family and training style without claiming to reproduce a full-scale industrial benchmark.

Below I provide the links to both code tutorials and walk through each in the remainder of the blog.

Notebook 1: 0D KPI Prediction (link here)

The first notebook uses the lightweight CSV files. Each row corresponds to a geometry and angle of attack, and the target is a

small vector of integrated aerodynamic coefficients:

inputs: geometry design variables + angle of attack
outputs: CL, CD, CM

The notebook starts with basic EDA: coefficient distributions, lift and drag trends with angle of attack, correlation structure, and drag polars. This matters because beginners should see the aerodynamic shape of the data before training a model. Lift rises with angle of attack, drag increases, and high-angle behavior becomes more nonlinear.

The model setup is intentionally simple. The geometry design variables are represented as a small set of tokens, and angle of attack is passed as a condition. Transolver++ then predicts the integrated coefficients. This is not a true field surrogate, but it is a useful first rung on the ladder:

- It runs quickly.
- It teaches dataset access and normalization.
- It introduces geometry-conditioned prediction.
- It gives immediate plots: predicted-vs-true scatter, drag polar comparisons, and coefficient curves.

The split is also important. Rather than randomly splitting rows, the notebook holds out geometries. That is more meaningful because the model must generalize to aircraft variants it did not see during training.

Notebook 2: 3D Point-Cloud Prediction (link here)

The second notebook moves from global coefficients to per-point prediction. This is closer to what people usually mean by an ML-CFD surrogate: given a geometry and condition, predict a flow quantity over a surface point cloud (and/or voluem field).

The serious data path uses the real boundary_*.vtu.tgz files from HiLiftAeroML. The notebook reads a VTU boundary field, samples surface points, extracts coordinates and normals, chooses a target field such as pressure coefficient or wall shear magnitude, and caches the reduced point cloud as a compact .npz.

The training target is now shaped like this:

inputs: sampled surface points + normals + AoA condition
outputs: scalar field value at each point

Because the raw files are huge, the notebook also includes a demo point-cloud mode. That mode creates small synthetic pressure-like point clouds so the training and visualization pipeline runs immediately. For a real project, you would switch to the real HiLiftAeroML boundary fields, preprocess several cases, and train on those cached point clouds. Serious models don't buckle under the pressure of >1M points per design, for scale.

The visual payoff is much better than the KPI model. The notebook plots:

- ground-truth surface point cloud
- predicted scalar field
- absolute error over the same geometry
- 2D projected views for easier inspection

This is the point where beginners start to build intuition for field surrogates: the model is not just predicting "one lift number"; it is learning a spatial distribution over an aircraft surface.

What This Example Is, and Is Not

This is an easy example on purpose. If I made it industrial grade then folks would need to pay for storage/GPUs, wait for ~24 hours for

trainings, and in general commit more time into running the examples. Considering this is meant for beginners, that would be a miss in my book. However, the framework is setup to easily scale from this toy example to something more serious (start by increasing the training samples, reducing the downsampling, training for more epochs, and running on better/newer GPUs which have more VRAM).

The 0D notebook uses small CSV summaries. The point-cloud notebook downsamples heavily and starts with a small number of cases. The Transolver++ model is kept small enough for Colab. The training loops are written for clarity, not maximum performance.

That is the right tradeoff for a first tutorial. A beginner should be able to run the code, see the data, understand the inputs and outputs, and get a reasonable fit before worrying about distributed training, multi-GPU data pipelines, mesh partitioning, or publication-grade benchmarks, etc.

But it also means the results should be interpreted honestly. These notebooks are starting points, not rigorous.

Making It Serious

The natural next steps are clear:

- Train on real boundary_*.vtu.tgz point clouds instead of the demo point-cloud mode.
- Use many geometry/AoA cases, not one or two. Separate cases into training and evaluation based on different criteria (i.e. force the more to progressively extrapolate while maintaining accuracy).
- Follow official train/validation/test splits if using the dataset for comparison.
- Predict physically meaningful surface fields such as Cp, wall shear stress, or multiple variables jointly.

- Recover integrated CL/CD/CM from predicted pressure/shear fields and compare against force_mom_all.csv.
- Add baselines: MLP, PointNet, graph neural network, MeshGraphNet-style model, or interpolation.
- Evaluate geometry generalization and AoA interpolation separately.
- Track error by aircraft component: slat, main wing, flap, tip, and junction regions.
- Add uncertainty estimates or ensembles, especially near stall and separated-flow regimes.
- Build a reusable preprocessing pipeline for VTU-to-point-cloud conversion.

For a resume or research portfolio, the stronger version of this project is not “I trained a model on a CFD dataset.” It is something more specific:

> *Built a Transolver++ point-cloud surrogate for NASA CRM-HL high-lift CFD data, including VTU preprocessing, geometry-conditioned training, surface-field visualization, and coefficient recovery from predicted pressure fields.*

For a publishable direction, the contribution would need to be more systematic: a curated reduced HiLiftAeroML benchmark, reproducible splits, strong baselines, field-level and coefficient-level metrics, and careful analysis of where the model fails.

Thanks for being here! GLHF.

Subscribed